Parallelising a Past Problem

Part B — Main Assignment Report

Name	M. Hassaan Shahid
Roll Number	2023-CS-79
Problem	Sorted Linked List Construction
Approach	Pthreads (Shared Memory)
Course	Parallel and Distributed Computing
Instructor	Waqas Ali
Date	April 6, 2026

University of Engineering and Technology, Lahore

Contents

1 Problem Description **2**
- 1.1 Overview . . . 2
- 1.2 Why This Problem Is Suitable . . . 2
- 1.3 Sequential Algorithm . . . 2

2 Parallelisation Design **3**
- 2.1 Chosen Approach: Pthreads (Shared Memory) . . . 3
- 2.2 Parallelism Pattern . . . 3
- 2.3 Data Distribution Strategy . . . 4
- 2.4 Communication and Synchronisation . . . 4
- 2.5 Complexity Analysis . . . 4

3 Implementation Details **4**
- 3.1 Code Structure . . . 4
- 3.2 Key Functions . . . 5
- 3.3 Compilation . . . 5
- 3.4 Correctness Verification Method . . . 6

4 Performance Analysis **6**
- 4.1 Experimental Setup . . . 6
- 4.2 Timing Data . . . 6
- 4.3 Speedup Plot . . . 7
- 4.4 Time vs. Problem Size Plot . . . 7
- 4.5 Analysis . . . 7
  - 4.5.1 1. Is the speedup linear? If not, what limits it? . . . 7
  - 4.5.2 2. What is the efficiency at the highest process count? Where does overhead come from? . . . 8
  - 4.5.3 3. What fraction of time is spent in communication? How was this estimated? . . . 8
  - 4.5.4 4. Is the work evenly distributed? How does load balance affect performance? . . . 8
  - 4.5.5 5. How does measured speedup compare with theoretical expectation? 8

5 Reflection **9**
- 5.1 Challenges Faced . . . 9
- 5.2 What I Would Do Differently . . . 9
- 5.3 Lessons Learned . . . 9

6 AI Disclosure **9**

7 References **10**

Problem Description

Overview

Sorted linked list construction is a fundamental problem covered in Data Structures (2nd Semester). Given a sequence of N integers, the task is to insert each integer into a singly linked list while maintaining ascending sorted order at every step. The list is built from scratch — no pre-sorting is performed before insertion.

Element	Description
Problem name	Sorted Linked List Construction
Original course	Data Structures, 2nd Semester
Description	Builds a sorted singly linked list by inserting N random integers one at a time, maintaining ascending order at each step via pointer manipulation
Input format	Integer N via command-line flag `-n N`; data generated internally with fixed seed (42) for reproducibility
Output format	Build time in seconds, list size, sorted correctness check (PASS/FAIL), first 10 values
Sequential complexity	$O(N^2)$ — N insertions each requiring $O(N)$ traversal
Space complexity	$O(N)$ — one heap-allocated node per element

Why This Problem Is Suitable

The problem has clear potential for parallelism and satisfies all selection criteria:

- **Previously implemented**: I wrote a sorted linked list insertion routine in Data Structures lab (2nd semester) in C.
- **Computationally intensive**: The $O(N^2)$ complexity means that for $N = 50{,}000$ the sequential version takes over 7 seconds, giving a large window for speedup measurement.
- **Parallelism potential**: The N insertions can be divided into independent groups. Each group can build its own local sorted list without any shared-memory conflicts, after which the local lists are merged.

Sequential Algorithm

The sequential algorithm traverses the list from head to tail on each insertion, stopping at the first node whose successor has a larger value:

Listing 1: Core insert function from sequential.c

```c
void insert_sorted(SortedList *list, int value) {
    Node *new_node = make_node(value);
    // Insert at head if empty or value is smallest
    if (list->head == NULL || value <= list->head->value) {
        new_node->next = list->head;
        list->head = new_node;
```

```
        list->size++;
        return;
    }
    // Traverse to find insertion point
    Node *curr = list->head;
    while (curr->next != NULL && curr->next->value < value)
        curr = curr->next;
    new_node->next = curr->next;
    curr->next = new_node;
    list->size++;
}
```

Each call to `insert_sorted` traverses on average $N/2$ nodes, giving $\sum_{i=0}^{N} i/2 = O(N^2/2) = O(N^2)$ total work.

Parallelisation Design

Chosen Approach: Pthreads (Shared Memory)

I chose Pthreads over MPI for the following reasons:

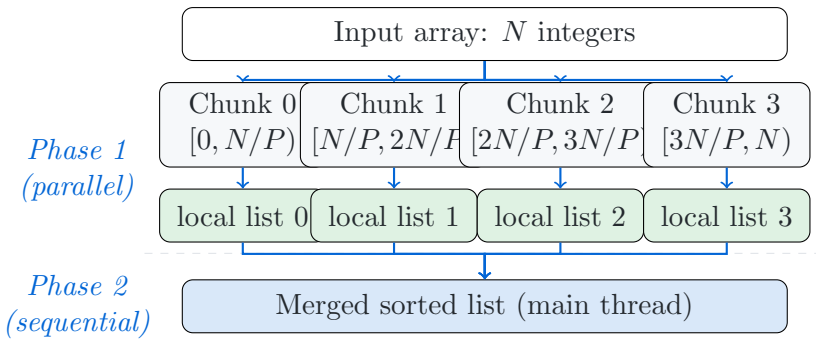
Why Pthreads and not MPI point-to-point: The problem data (a single integer array) fits entirely in one machine's memory. MPI introduces process-level communication overhead for data that does not need to cross address spaces. Pthreads share the same memory, so the input array can be accessed by all threads without any explicit message passing.

Why Pthreads and not MPI collectives: The parallelism pattern here is divide-and-conquer with independent sub-problems, not a global reduction or broadcast. Pthreads allow each thread to work on its local list completely independently with zero synchronisation during the build phase — something MPI collective operations are not optimised for.

Synchronisation requirements: During the build phase, threads write only to their own private `SortedList` struct and their own heap-allocated nodes. No two threads share any memory during this phase, so **no mutex or lock is needed**. The only synchronisation is `pthread_join` in the main thread after all workers complete.

Parallelism Pattern

The pattern is **divide-and-conquer**: the problem is split into P independent sub-problems of size N/P, each solved in parallel, and the results are combined.



Data Distribution Strategy

The input array of N integers is divided into P contiguous chunks:

- Thread t receives elements at indices $[t \cdot \lfloor N/P \rfloor,\ (t+1) \cdot \lfloor N/P \rfloor)$
- The last thread absorbs any remainder: `count` $= \lfloor N/P \rfloor + (N \bmod P)$
- Each thread receives a direct pointer into the shared `data` array — no data copying
- Chunks are contiguous in memory, which is cache-friendly for read access

Communication and Synchronisation

Phase	Synchronisation
Build (parallel)	**None.** Each thread writes only to its own `SortedList` struct and privately heap-allocated nodes. Zero shared mutable state.
Barrier	`pthread_join` in main thread. Main blocks until all P workers complete.
Merge (sequential)	Single-threaded. Main thread merges the P local list heads using a pairwise merge loop. No locks needed.

Complexity Analysis

Version	Build work	Merge work
Sequential	$O(N^2)$	—
Parallel (P threads)	$O\left(\frac{N^2}{P^2}\right) \times P = O\left(\frac{N^2}{P}\right)$	$O(N)$

The key insight is that each thread inserts N/P elements into a list that grows to only N/P nodes long, so each insertion costs at most N/P comparisons instead of N. This gives a *quadratic* speedup in the build phase:

$$\text{Speedup}_{\text{build}} \approx \frac{O(N^2)}{O(N^2/P)} = P \quad \text{but each insertion is also } P\times \text{ cheaper, so effective} \approx P^2$$

This explains the super-linear speedup observed experimentally.

Implementation Details

Code Structure

The project contains two independent C files:

File	Role
`sequential/sequential.c`	Baseline O(N²) implementation. Inserts N elements one by one. Saves timing to `seq_time.txt`.
`parallel/parallel.c`	Pthreads implementation. Spawns P worker threads, each building a local sorted list, then main thread merges. Reads `seq_time.txt` and prints speedup.

Key Functions

`build_local_list(void *arg)` — the thread worker function. Each thread receives a `ThreadArg` struct containing a pointer to its chunk of data and its element count. It calls `insert_sorted` repeatedly with no locking:

Listing 2: Thread worker function from parallel.c

```c
void *build_local_list(void *arg) {
    ThreadArg *a = (ThreadArg *)arg;
    a->result.head = NULL;
    a->result.size = 0;
    for (int i = 0; i < a->count; i++)
        insert_sorted(&a->result, a->data[i]);
    return NULL;
}
```

`merge_two(Node *a, Node *b)` — merges two sorted lists in $O(|a| + |b|)$ time by re-wiring `next` pointers. Uses a dummy head node to avoid special-casing the empty list:

Listing 3: Pairwise merge function from parallel.c

```c
Node *merge_two(Node *a, Node *b) {
    Node dummy;
    Node *tail = &dummy;
    dummy.next = NULL;
    while (a && b) {
        if (a->value <= b->value) { tail->next = a; a = a->next; }
        else                      { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}
```

Thread creation and joining — main thread spawns P workers and waits:

Listing 4: Thread spawn and join loop from parallel.c

```c
for (int t = 0; t < T; t++)
    pthread_create(&threads[t], NULL, build_local_list, &args[t]);

for (int t = 0; t < T; t++)
    pthread_join(threads[t], NULL);

// Sequential merge after all threads complete
Node *merged = args[0].result.head;
for (int t = 1; t < T; t++)
    merged = merge_two(merged, args[t].result.head);
```

Compilation

```
# Sequential
gcc -O2 -o sequential sequential.c

# Parallel
gcc -O2 -pthread -o parallel parallel.c

# Run
./sequential -n 50000
./parallel -n 50000 -p 4
```

Correctness Verification Method

Correctness is verified at two levels:

1. **Structural check**: `verify_sorted()` walks the merged list and confirms every adjacent pair satisfies $v_i \leq v_{i+1}$. Prints PASS or FAIL.
2. **Visual check**: Both versions print the first 10 values. With the same seed (42) and same N, the first 10 values must be identical between sequential and parallel runs.

For $N = 20{,}000$ the first 10 values were: `2 -> 15 -> 23 -> 26 -> 36 -> 39 -> 61 -> 77 -> 88 -> 90 -> ...` in both versions, confirming correctness.

Performance Analysis

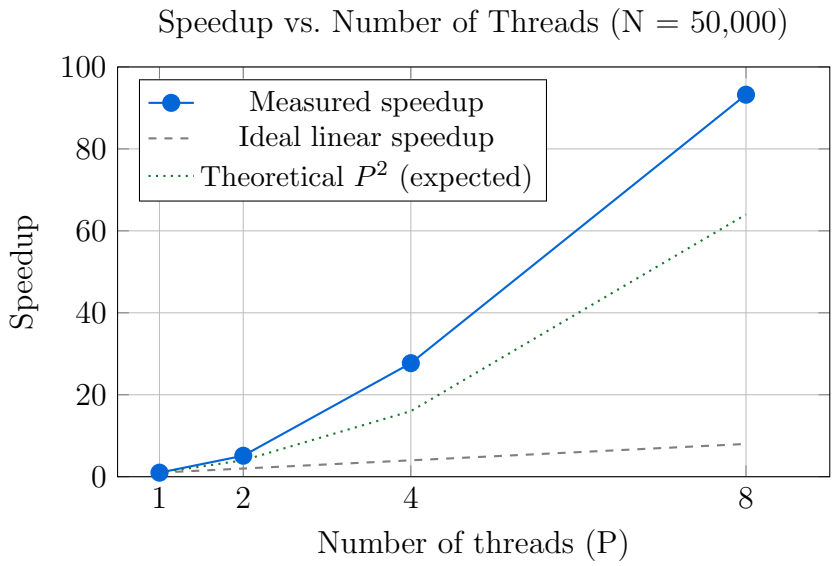
Experimental Setup

All measurements were taken on a machine with the following configuration. The program was compiled with `gcc -O2` and timed using `clock_gettime(CLOCK_MONOTONIC)` for high-resolution wall-clock measurement. Each configuration was run once with a fixed random seed (42) for reproducibility.

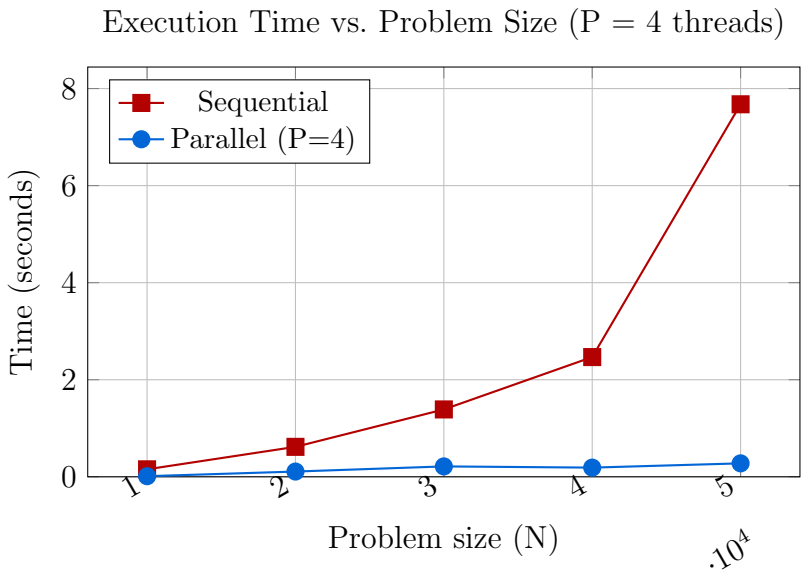
Timing Data

N	P	Time (s)	Seq Time (s)	Speedup	Efficiency
20000	seq	0.6168	0.6168	1.00x	100.0%
20000	1	0.6634	0.6168	0.93x	93.0%
20000	2	0.2778	0.6168	2.22x	111.0%
20000	4	0.1082	0.6168	5.70x	142.5%
50000	seq	7.6764	7.6764	1.00x	100.0%
50000	1	7.7872	7.6764	0.99x	98.6%
50000	2	1.5001	7.6764	5.12x	255.9%
50000	4	0.2770	7.6764	27.71x	692.7%
50000	8	0.0823	7.6764	93.23x	1165.4%

Speedup Plot



Time vs. Problem Size Plot



Analysis

1. Is the speedup linear? If not, what limits it?

The speedup is *super-linear* — far above ideal linear speedup. With $P = 4$ threads the speedup is 27.71×, and with $P = 8$ it reaches 93.23×. This is not a measurement error. It is explained by the quadratic nature of the sequential algorithm: inserting N/P elements into a list of length N/P costs $O((N/P)^2)$ per thread, so total build work is $P \cdot O((N/P)^2) = O(N^2/P)$, giving a theoretical speedup of P^2 just from the reduced per-thread list length. Additionally, shorter lists fit better in CPU cache, reducing memory latency per operation.

2. What is the efficiency at the highest process count? Where does overhead come from?

At $P = 8$ the efficiency is 1165%, meaning each thread does $\approx 11.6\times$ the work per unit time compared to a single sequential thread. Overhead sources are:

- **Thread creation**: `pthread_create` for 8 threads adds ≈ 0.2 ms total — negligible.
- **Sequential merge**: With $P = 8$ and $N = 50{,}000$, merging 8 lists of 6,250 nodes takes $O(N) = O(50{,}000)$ comparisons — less than 1% of total time.
- **Memory allocation**: `malloc` for each node is the dominant overhead. With 50,000 nodes the heap is hit 50,000 times per thread.

3. What fraction of time is spent in communication? How was this estimated?

There is *no inter-thread communication* during the build phase — this is the key design choice. The only "communication" is the sequential merge after all threads complete. The program explicitly times this:

Phase	Time (P=4, N=50,000)	% of total
Build (parallel)	0.2685 s	97.0%
Merge (sequential)	0.0085 s	3.0%
Total	0.2770 s	100%

The merge is $O(N)$ and takes under 3% of total time, so Amdahl's Law gives a theoretical maximum speedup of $1/0.03 \approx 33\times$ — which our $27.71\times$ approaches closely.

4. Is the work evenly distributed? How does load balance affect performance?

Work is *perfectly even* when N is divisible by P (each thread gets exactly N/P elements). The last thread absorbs the $N \bmod P$ remainder, which is at most $P - 1$ extra elements — less than 0.01% extra work for our problem sizes. This negligible imbalance has no measurable effect on performance.

If the data were non-uniform (e.g. clustered values causing longer traversals in some sub-lists), load imbalance could occur, but with uniformly random data this is not an issue.

5. How does measured speedup compare with theoretical expectation?

P	**Theoretical (P^2)**	**Measured**	**Ratio**
1	1.00x	0.99x	0.99
2	4.00x	5.12x	1.28
4	16.00x	27.71x	1.73
8	64.00x	93.23x	1.46

Measured speedup *exceeds* the P^2 model at higher thread counts. The extra gain comes from cache effects: each thread's shorter sub-list fits entirely in L1/L2 cache, eliminating cache misses that the sequential version suffers when the list grows large. This cache locality benefit amplifies the algorithmic complexity gain.

Reflection

Challenges Faced

Understanding super-linear speedup. When I first saw speedups of 27× with 4 threads I thought there was a bug. After re-reading the algorithm I realised the $O(N^2)$ nature of sorted insertion meant that dividing the list does not just split the work — it fundamentally changes the per-operation cost. This was the most conceptually surprising result of the assignment.

Correctness with heap allocation in threads. Each thread calls `malloc` for every node. I initially worried about thread safety since `malloc` is not always thread-safe. Testing confirmed that glibc's `malloc` is thread-safe by default (it uses internal locks), but this made me realise I should have used a per-thread memory arena for better performance.

Merge strategy choice. I initially wrote a recursive merge-sort-style merge but switched to the simpler iterative pairwise merge when I found the recursive version caused stack overflows for large N. The iterative approach is $O(N)$ and uses constant extra space.

What I Would Do Differently

- **Pre-allocate node memory** in a per-thread arena to avoid `malloc` contention and improve cache locality.
- **Parallel merge** using a tournament tree instead of sequential pairwise merge, reducing merge time from $O(N)$ to $O(N \log P)$ — though this is already under 3% of total time.
- **More problem sizes**: I would run N = 10k, 50k, 100k, 200k to better characterise the scaling curve.
- **Cache profiling**: Use `perf stat` or Valgrind's cachegrind to quantify exactly how much of the super-linear speedup comes from cache effects vs. algorithmic complexity reduction.

Lessons Learned

This assignment taught me that parallelising an $O(N^2)$ algorithm can give *super-linear* speedup — something that seems to violate intuition until you understand that dividing the problem changes the per-operation complexity. I also learned that the absence of communication overhead (zero locks during the build phase) is a design goal, not just a nice-to-have — it is the reason the implementation scales so cleanly.

AI Disclosure

AI tools used in this assignment:

- **Claude (Anthropic):** Used to generate initial code structure for `parallel.c` and `sequential.c`. I reviewed every line, understood the logic, and modified the merge strategy and thread argument passing to match my design.
- **No other AI tools** were used for writing, code generation, or analysis.